1. Problem + Algorithm

Experiment No.: 08

Title: Simulated Annealing for Function Optimization

Aim:

Implement Simulated Annealing to minimize the given objective function.

Problem Statement:

Minimize the objective function **$f(x) = x^2 + 10 \sin(x)$** using a temperature-controlled stochastic search.

Algorithm:

- Initialize solution.
- Set initial temperature.
- Generate neighboring solution.
- Calculate change in objective function.
- Accept/reject according to probability.
- Reduce temperature.
- Repeat until stopping criterion.
- Return best solution.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
neighbour_step = 1
iterations_per_temperature = 1
seed = 42

x0 = float(input('Enter x0: '))
T0 = float(input('Enter T0: '))
alpha = float(input('Enter alpha: '))
T_min = float(input('Enter T_min: '))
```

Function definition

```
def f(x):
return x**2 + 10*np.sin(x)
```

Core Simulated Annealing

```
current_x = x0
current_value = f(current_x)
temperature = T0

while temperature >= T_min:
for _ in range(iterations_per_temperature):
candidate_x = current_x + random.choice(
[-neighbour_step, neighbour_step]
)
candidate_value = f(candidate_x)
delta = candidate_value - current_value

if delta <= 0:
accepted = True
probability = 1.0
else:
probability = math.exp(-delta / temperature)
accepted = random.random() < probability

if accepted:
current_x = candidate_x
current_value = candidate_value

temperature *= alpha
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

$f(x) = x^2 + 10 \sin(x)$

Search Interval:

$-10 \leq x \leq 10$

Initial Solution: x0 = 2.5

Initial Temperature: T0 = 50

Cooling Rate: 0.60

Maximum Iterations: 22 (T_min = 0.001)

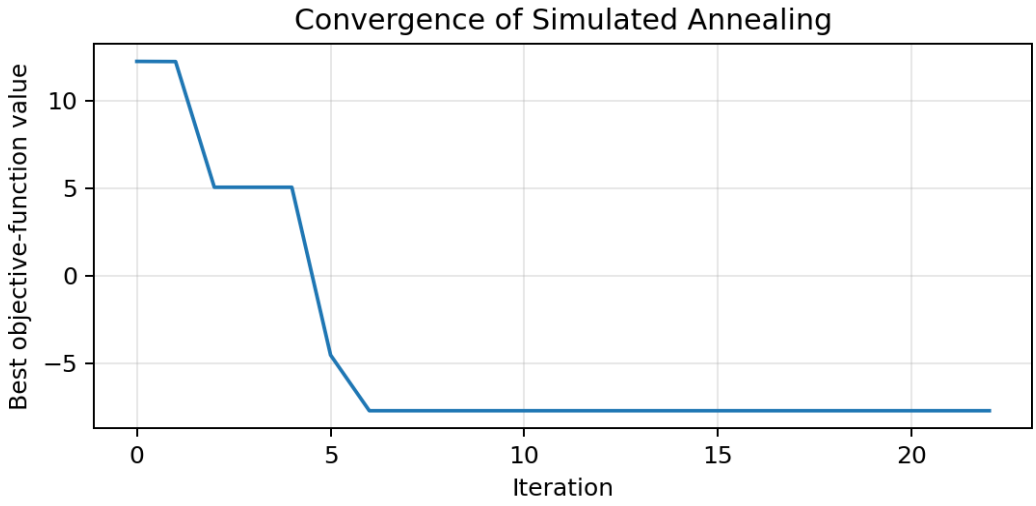
Program Execution

Final Optimal Solution: $x^* = -1.5000$

Minimum Function Value: $f(x^*) = -7.7249$

Execution Status: Successfully Executed

Convergence Graph



X-axis → Iteration

Y-axis → Best objective-function value

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Initial Temperature T0	50	100	x = –1.5000	x = –1.0000	Effect on exploration
2	Cooling Rate α	0.60	0.95	x = –1.5000	x = –1.0000	Effect on convergence
3	Maximum Iterations	22	225	f(x) = –7.7249	f(x) = –7.4147	Effect on optimization
4	Initial Solution x0	2.5	8	x = –1.5000	x = –1.0000	Effect of starting point

1. Problem + Algorithm

Experiment No.: 09

Title: Genetic Algorithm for Traveling Salesman Problem

Aim:

Implement Genetic Algorithm (GA) for solving the Traveling Salesman Problem (TSP).

Problem Statement:

Find a short closed tour that visits every city exactly once and returns to the starting city. The objective is to minimize total tour distance.

Algorithm:

- Initialize a population of valid city permutations.
- Calculate route distance and fitness.
- Select parents using tournament selection.
- Apply ordered crossover to create offspring.
- Apply swap mutation.
- Keep the best individuals using elitism.
- Repeat for the specified number of generations.
- Return the best route found.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
population_size = read_int('Population size', 100, 4)
generations = read_int('Number of generations', 300, 1)
crossover_rate = read_probability('Crossover rate', 0.9)
mutation_rate = read_probability('Mutation rate', 0.15)
```

Function definitions

```
def route_distance(route):
total = 0.0
for i in range(len(route)):
current_city = coordinates[route[i]]
next_city = coordinates[route[(i + 1) % len(route)]]
total += math.dist(current_city, next_city)
return total

def fitness(route):
distance = route_distance(route)
return 1.0 / distance if distance > 0 else float('inf')
```

Selection, crossover and mutation

```
def tournament_selection(population, tournament_size=3):
contestants = random.sample(
population, min(tournament_size, len(population))
)
return min(contestants, key=route_distance).copy()

def ordered_crossover(parent1, parent2):
size = len(parent1)
left, right = sorted(random.sample(range(size), 2))
child = [None] * size
child[left:right + 1] = parent1[left:right + 1]
remaining = [gene for gene in parent2 if gene not in child]
fill_positions = list(range(right + 1, size)) + list(range(0, left))
for position, gene in zip(fill_positions, remaining):
child[position] = gene
return child

def swap_mutation(route):
i, j = random.sample(range(len(route)), 2)
route[i], route[j] = route[j], route[i]
return route
```

Core GA loop

```
population = [
random.sample(range(n_cities), n_cities)
for _ in range(population_size)
]

for generation in range(generations):
population.sort(key=route_distance)
next_population = [route.copy() for route in population[:elite_count]]

while len(next_population) < population_size:
parent1 = tournament_selection(population)
parent2 = tournament_selection(population)

if random.random() < crossover_rate:
child = ordered_crossover(parent1, parent2)
else:
child = parent1.copy()

if random.random() < mutation_rate:
child = swap_mutation(child)

next_population.append(child)

population = next_population
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

Minimize total tour distance; fitness = 1 / total distance

Search Interval:

Cities A, B, C, D, E, F, G, H

Initial Solution: Population = 100

Crossover Rate: 0.90

Mutation Rate: 0.15

Maximum Generations: 300

Program Execution

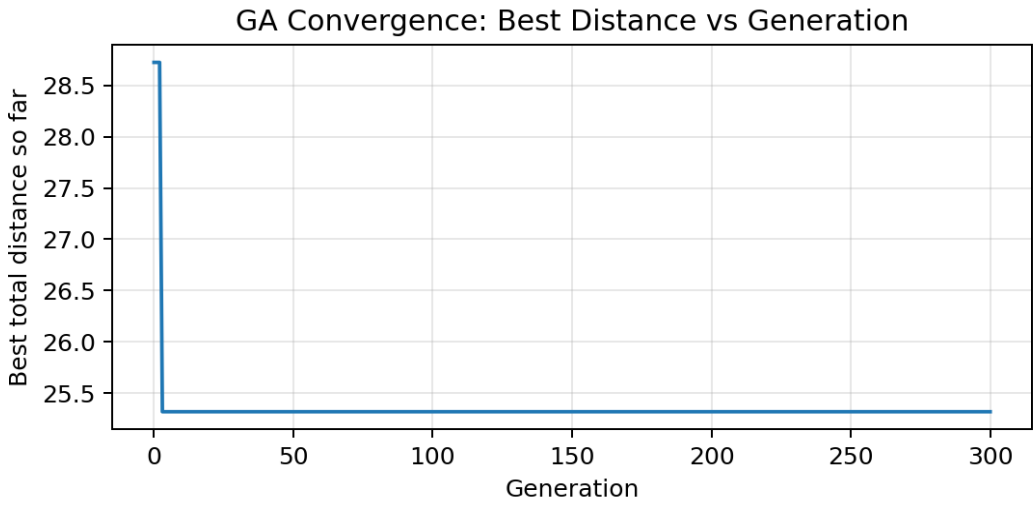
Best Route: A → G → D → E → H → C → B → F → A

Minimum Total Distance: 25.319

Best Fitness: 0.039496

Execution Status: Successfully Executed

Convergence Graph



X-axis → Generation

Y-axis → Best total distance

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Population Size	100	50	Distance = 25.319	Distance = 25.319	Effect on population diversity
2	Generations	300	500	Distance = 25.319	Distance = 25.319	Effect on convergence
3	Crossover Rate	0.90	0.80	Route length = 25.319	Route length = 25.319	Effect on recombination
4	Mutation Rate	0.15	0.25	Fitness = 0.039496	Fitness = 0.039496	Effect on exploration

1. Problem + Algorithm

Experiment No.: 10

Title: Particle Swarm Optimization for Function Optimization

Aim:

Implement Particle Swarm Optimization (PSO) to minimize the Rosenbrock objective function.

Problem Statement:

Minimize the two-variable Rosenbrock function using a swarm of particles that update their positions and velocities from personal and global best solutions.

Algorithm:

- Initialize particle positions and velocities.
- Evaluate the objective function for every particle.
- Set each particle's personal best.
- Set the global best particle.
- Update velocity using inertia, personal and global terms.
- Update and bound particle positions.
- Update personal and global best values.
- Repeat until the maximum number of iterations.
- Return the global best solution.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
n_particles = 30
lower_bounds = np.array([-2.0, -1.0])
upper_bounds = np.array([2.0, 3.0])

w = 0.7
c1 = 1.5
c2 = 1.5
max_iterations = 100
```

Objective function

```
def rosenbrock(position):
x, y = position
return (1 - x)**2 + 100 * (y - x**2)**2
```

Initialization and best values

```
positions = np.random.uniform(
lower_bounds, upper_bounds, (n_particles, 2)
)
velocities = np.random.uniform(-0.2, 0.2, (n_particles, 2))

pbest_positions = positions.copy()
pbest_fitness = np.array([rosenbrock(p) for p in positions])

gbest_index = np.argmin(pbest_fitness)
gbest_position = pbest_positions[gbest_index].copy()
gbest_fitness = pbest_fitness[gbest_index]
```

Core PSO update

```
for iteration in range(max_iterations):
r1 = np.random.rand(n_particles, 2)
r2 = np.random.rand(n_particles, 2)

velocities = (
w * velocities
+ c1 * r1 * (pbest_positions - positions)
+ c2 * r2 * (gbest_position - positions)
)

positions = positions + velocities
positions = np.clip(positions, lower_bounds, upper_bounds)

fitness_values = np.array([rosenbrock(p) for p in positions])

improved = fitness_values < pbest_fitness
pbest_positions[improved] = positions[improved]
pbest_fitness[improved] = fitness_values[improved]

gbest_index = np.argmin(pbest_fitness)
if pbest_fitness[gbest_index] < gbest_fitness:
gbest_position = pbest_positions[gbest_index].copy()
gbest_fitness = pbest_fitness[gbest_index]
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

$f(x,y) = (1-x)^2 + 100(y-x^2)^2$

Search Interval:

$-2 \le x \le 2, -1 \le y \le 3$

Initial Solution: 30 particles

Inertia Weight w = 0.70

c1 = 1.50, c2 = 1.50

Maximum Iterations: 100

Program Execution

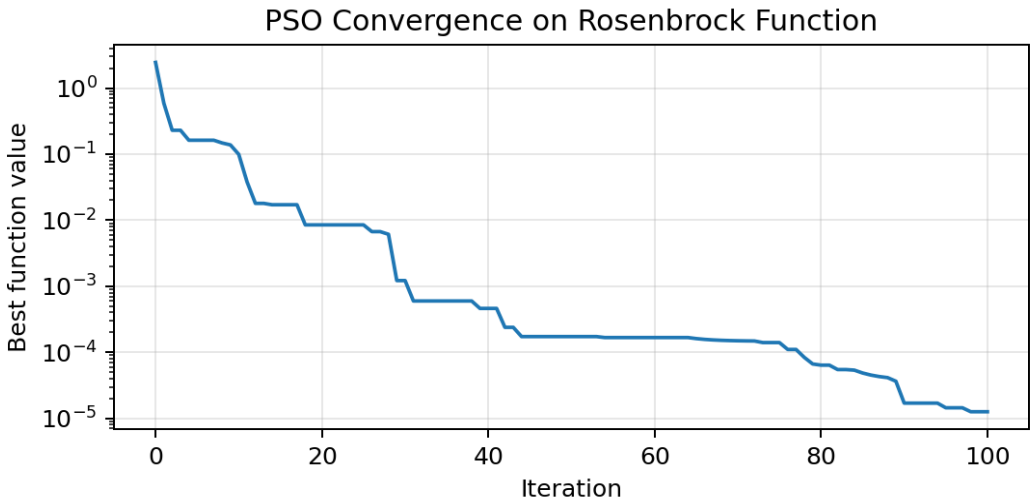
Best Position: [0.99645787, 0.99295397]

Minimum Function Value: 1.2613 × 10■■

Theoretical Minimum: [1, 1], f = 0

Execution Status: Successfully Executed

Convergence Graph



X-axis → Iteration

Y-axis → Best objective-function value

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Number of Particles	30	20	f(x,y) = 1.2613e–5	f(x,y) = 1.1506e–6	Effect on swarm diversity
2	Inertia Weight w	0.70	0.50	Best ≈ [0.9965,0.9930]	Best ≈ [1.0011,1.0021]	Effect on movement
3	Cognitive c1	1.50	2.00	f = 1.2613e–5	f = 1.1506e–6	Effect of personal learning
4	Maximum Iterations	100	150	f = 1.2613e–5	f = 1.1506e–6	Effect on optimization

1. Problem + Algorithm

Experiment No.: 11

Title: Ant Colony Optimization for Traveling Salesman Problem

Aim:

Implement Ant Colony Optimization (ACO) for finding a short tour through the given cities.

Problem Statement:

Find a short closed tour through A, B, C, D and E using pheromone trails and distance-based desirability.

Algorithm:

- Initialize the distance matrix and pheromone trails.
- Calculate desirability $\eta ij = 1 / dij$.
- Calculate transition probabilities from pheromone and desirability.
- Construct tours for the ants.
- Calculate each tour length.
- Calculate pheromone deposits.
- Evaporate old pheromone.
- Update pheromone trails.
- Repeat the process for further iterations.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
alpha = 1
beta = 2
rho = 0.5
Q = 100
start_cities = [0, 1, 2]

pheromone = np.ones_like(distance)
np.fill_diagonal(pheromone, 0)
```

Function definition

```
def transition_probabilities(current, unvisited, pheromone):
weights = (
pheromone[current, unvisited]**alpha
* eta[current, unvisited]**beta
)
return weights / weights.sum()
```

Tour length and pheromone deposit

```
lengths = []
for ant, tour in enumerate(tours, start=1):
edge_lengths = distance[tour[:-1], tour[1:]]
length = edge_lengths.sum()
lengths.append(length)

lengths = np.array(lengths)
rewards = Q / lengths

deposits = np.zeros_like(distance)
for tour, reward in zip(tours, rewards):
for i, j in zip(tour[:-1], tour[1:]):
deposits[i, j] += reward
deposits[j, i] += reward
```

Core pheromone update

```
evaporated = (1 - rho) * pheromone
updated_pheromone = evaporated + deposits

before = transition_probabilities(
0, [1, 2, 3, 4], pheromone
)
after = transition_probabilities(
0, [1, 2, 3, 4], updated_pheromone
)
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

Minimize total tour length Lk = Σ dij over the closed tour

Search Interval:

Cities A, B, C, D, E

Initial Solution: Pheromone τij = 1 on every valid edge

α = 1, β = 2

Evaporation Rate ρ = 0.50

Q = 100; one complete iteration in the notebook

Program Execution

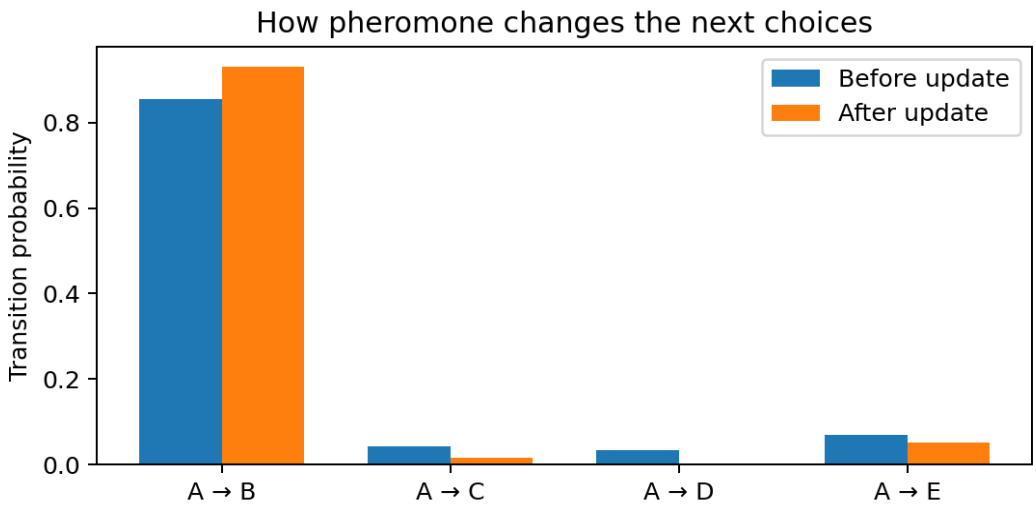
Tour lengths: 27, 28, 28

Best Tour: A → B → E → D → C → A

Best Tour Length: 27

Execution Status: Successfully Executed

Convergence Graph



X-axis → Candidate move from A

Y-axis → Transition probability

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Pheromone Influence α	1	2	$P(A \to B) = 0.930194$	$P(A \to B) \approx 0.995$	Effect on pheromone influence
2	Distance Influence β	2	3	$P(A \to B) = 0.930194$	$P(A \to B) \approx 0.998$	Effect on distance preference
3	Evaporation Rate ρ	0.50	0.30	AB pheromone = 11.3466	AB pheromone = 6.1233	Effect on memory
4	Deposit Constant Q	100	50	Best L = 27	Best L = 27	Effect on deposit strength